

Coursework Answers
Probabilistic Generative Models
CID:06060027

Part 1: Bernoulli Mixture on Binarized MNIST – Report

This section applies the EM framework to a Bernoulli mixture. Data preparation and experiments are implemented in the companion script `solutions/part1/bernoulli_mixture.py` (digits 0, 1, 8, thresholded at 0.5).

Task 1 (complete-data log-likelihood, parameters, latent assignment, E/M updates). For $x_n \in \{0, 1\}^D$, define latent assignment variables $z_n = (z_{n1}, \dots, z_{nK})$, $z_{nk} \in \{0, 1\}$, $\sum_k z_{nk} = 1$. The model parameters are

$$\theta = \{\pi_1, \dots, \pi_K, \mu_{11}, \dots, \mu_{KD}\},$$

with $\pi_k \geq 0$, $\sum_k \pi_k = 1$, $\mu_{kd} \in (0, 1)$. The complete-data model is

$$p(x_n, z_n \mid \pi, \mu) = \prod_{k=1}^K \left[\pi_k \prod_{d=1}^D \mu_{kd}^{x_{nd}} (1 - \mu_{kd})^{1-x_{nd}} \right]^{z_{nk}}.$$

Hence

$$\log p(X, Z \mid \pi, \mu) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \left[\log \pi_k + \sum_{d=1}^D (x_{nd} \log \mu_{kd} + (1 - x_{nd}) \log(1 - \mu_{kd})) \right].$$

E-step responsibilities:

$$\gamma_{nk} := p(z_{nk} = 1 \mid x_n, \pi, \mu) = \frac{\pi_k \prod_{d=1}^D \mu_{kd}^{x_{nd}} (1 - \mu_{kd})^{1-x_{nd}}}{\sum_{j=1}^K \pi_j \prod_{d=1}^D \mu_{jd}^{x_{nd}} (1 - \mu_{jd})^{1-x_{nd}}}.$$

M-step updates:

$$\pi_k^{(t+1)} = \frac{1}{N} \sum_{n=1}^N \gamma_{nk}^{(t)}, \quad \mu_{kd}^{(t+1)} = \frac{\sum_{n=1}^N \gamma_{nk}^{(t)} x_{nd}}{\sum_{n=1}^N \gamma_{nk}^{(t)}}.$$

Task 2 (why E-step in log-probability space). The E-step involves products of $D = 784$ Bernoulli factors in $(0, 1)$, which can underflow in floating-point arithmetic. Computing log-joint terms converts products into sums and keeps values numerically stable; subtracting the row-wise maximum before exponentiation gives a stable normalization for γ_{nk} .

Task 3 (hyperparameters, loss evolution, and choice of K). I tested different N, K combinations and plotted loss evolution (Figure 1). Numeric summary from `loss_by_experiment.csv`:

Experiment	N	K	Final NLL	Final NLL / obs
overcomplete	1500	5	221108.007	147.405
more_data	1500	3	239021.536	159.348
baseline	600	3	95662.090	159.437
undercomplete	1500	2	251475.361	167.650

Different settings provide different fit quality: $K = 5$ (overcomplete) gives the best training NLL, while $K = 2$ underfits. However, because the training set contains 3 unique digits (0, 1, 8), $K = 3$ is the semantically matched and most interpretable choice; $K > 3$ can improve training fit by splitting a true class into multiple mixture components. A useful diagnostic is to compare final NLL per observation against interpretability of the learned components. The overcomplete model improves training fit because additional components can represent intra-class style variation (for example, thin versus thick strokes) rather than truly new semantic classes. This flexibility is beneficial for density estimation, but it can reduce component identifiability and make class-to-component associations less stable across seeds. By contrast, $K = 3$ encourages one dominant component per digit and yields cleaner prototype interpretation. Increasing N from 600 to 1500 also reduces estimation variance and produces smoother, more stable loss trajectories. Overall, this supports $K = 3$ as a balanced default for interpretable reporting.

Task 4 (empirical convergence: monotonicity and sensitivity to initialization). The EM lower-bound identity

$$\log p(x \mid \theta) = \mathcal{L}(q, \theta) + \text{KL}(q \parallel p(z \mid x, \theta))$$

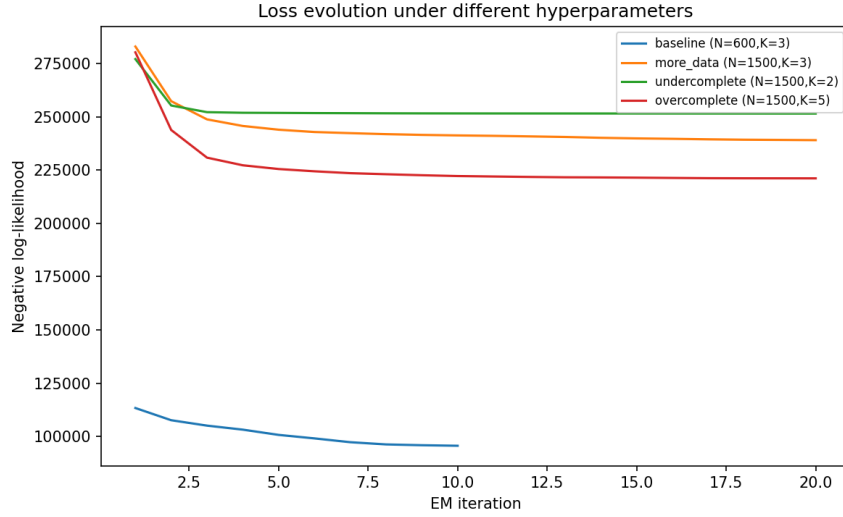


Figure 1: Part 1, Task 3–4: loss evolution for baseline, more-data, undercomplete, and overcomplete settings (from `loss_by_experiment.csv`).

supports monotone objective improvement under exact E-steps and M-step maximization. Empirically, monotonic NLL decrease held in 5/5 random seeds; final losses ranged 231373.512 to 239021.536, so convergence is monotone but the attained local optimum depends on initialization.

Task 5 (interpretation of learnt Bernoulli means). Each μ_k is a prototype image: pixel d in component k is the Bernoulli probability that that pixel is “on”. Responsibilities γ_{nk} provide soft membership of each sample to each prototype. Component-label associations are $0 \rightarrow 0$ (0.604), $1 \rightarrow 8$ (0.522), $2 \rightarrow 1$ (0.991).

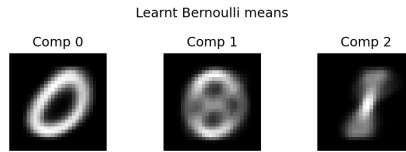


Figure 2: Part 1, Task 5: learned Bernoulli mean images μ_k .

Task 6 (sampling from the learnt model and quality analysis). I implemented `sample_bernoulli_mixture` in `bernoulli_mixture.py`: first sample a component $k \sim \text{Cat}(\pi)$, then sample each pixel independently from $\text{Bernoulli}(\mu_{kd})$. Generated images are recognizable (see $\alpha = 1$ panel in Figure 3) but blurrier/noisier than real digits because the model assumes conditional independence between pixels (no spatial structure) and uses a finite set of global prototypes.

Task 7 (temperature α and its effect). Using

$$\mu \mapsto \tilde{\mu} = \frac{\mu^\alpha}{\mu^\alpha + (1 - \mu)^\alpha}, \quad \alpha \geq 1,$$

pushes probabilities away from 0.5. Figure 3 ($\alpha = 1, 2, 4$) shows that larger α increases contrast and sharpness, but can reduce diversity as samples become more deterministic for large α .

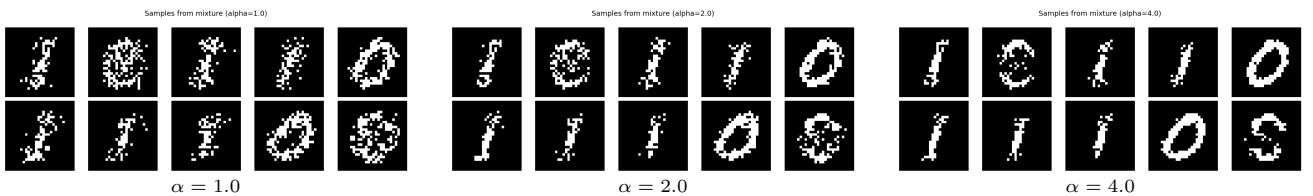


Figure 3: Part 1, Tasks 6–7: generated digits under temperature sharpening.

Implementation file included in submission: `solutions/part1/bernoulli_mixture_submission.py`. Reproduce Part 1 outputs with `python3 solutions/part1/bernoulli_mixture_submission.py (np.random.seed(0))`; sensitivity sweep uses seeds $0, \dots, 4$.

Part 2: Gaussian Process Regression – Report

This section presents Gaussian-process regression through Bayesian linear-model marginalization and marginal-likelihood training. Implementation file included in submission: `solutions/part2/gaussian.process.submission.py`.

Task 1 (implement `SE(x,y,theta)` and `Laplace(x,y,theta)`). The missing noiseless kernel functions were implemented as

$$k_{\text{SE}}(x, y) = \sigma^2 \exp\left(-\frac{(x - y)^2}{2\ell^2}\right), \quad k_{\text{Lap}}(x, y) = \sigma^2 \exp\left(-\frac{|x - y|}{2\ell^2}\right),$$

with $\theta = (\sigma^2, \ell, \sigma_e^2)$. Both are vectorised over all pairs (x_i, y_j) . Noise is added only by the provided `SE_wnoise` and `Laplace_wnoise` wrappers.

Task 2 (prior trajectories, hyperparameters, and range of the space variable). Using `sample_from_prior` and `plot_prior`, I generated trajectories for both kernels with different hyperparameters ($\ell = 25, 80$, $\sigma^2 = 5$, $\sigma_e^2 = 1$). The main plots use $x \in [0, 1750]$, and I also checked shorter and longer ranges. SE priors are smoother, while Laplace priors admit sharper local changes; increasing ℓ increases correlation range and smoothness for both kernels. To make the range effect explicit, I compared roughness statistics (mean absolute first difference of sampled trajectories) on $x \in [0, 500]$, $[0, 1750]$, and $[0, 3500]$. For SE, roughness at $\ell = 25$ was 0.1649, 0.2996, 0.3004, and at $\ell = 80$ it was 0.0525, 0.0900, 0.1004. For Laplace, roughness at $\ell = 25$ was 0.1103, 0.1499, 0.1501, and at $\ell = 80$ it was 0.0345, 0.0469, 0.0470. This is consistent with the plots: larger ranges expose more long-distance variation at fixed ℓ , while larger ℓ suppresses local oscillation.

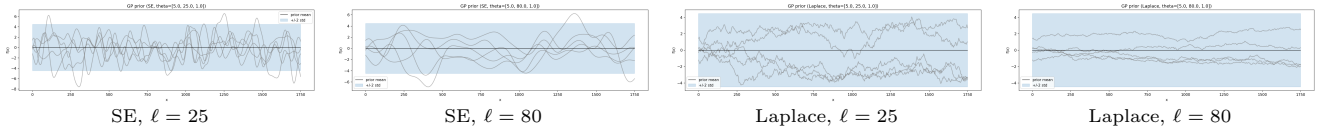


Figure 4: Part 2, Task 2: GP prior trajectories for both kernels and multiple hyperparameters.

Task 3 (implement `gp.nlogp(hypers)`). For observations $y \in \mathbb{R}^n$, the implemented objective is

$$\text{NLL}(\theta) = \frac{1}{2} y^\top K_\theta^{-1} y + \frac{1}{2} \log |K_\theta| + \frac{n}{2} \log(2\pi), \quad K_\theta = K + \sigma_e^2 I.$$

The implementation uses Cholesky-based linear algebra (`cho_factor`, `cho_solve`) and returns a large penalty for invalid/non-positive hyperparameters or failed factorizations.

Task 4 (test `gp.nlogp` on `hr2.txt`: initializations, optimizers, amounts of data). Using both kernels on the provided `hr2.txt` series, I varied optimizer, initialization, and training fraction. Results:

Kernel + optimizer	Final NLL	Success
SE + L-BFGS-B	431.567963	True
SE + Powell	431.568022	True
Laplace + L-BFGS-B	440.625	True
Laplace + Powell	451.242	True

At 10% observations with default initialization, SE obtains lower NLL than Laplace; Laplace is more sensitive to optimizer choice (Powell degrades more). The two SE values are almost identical but not exactly equal: they coincide at 3 decimal places because the gap is only $\approx 5.9 \times 10^{-5}$. This indicates both optimizers reached essentially the same basin of the objective. The resulting SE hyperparameters are also nearly the same (31.0455, 14.1606, 1.0798 for L-BFGS-B versus 31.1002, 14.1640, 1.0794 for Powell), which further supports practical equivalence of the fitted solution in this run.

Initial-condition sensitivity (10% observations, L-BFGS-B):

Kernel	Init	Final NLL
SE	small / default / large	431.568 / 431.568 / 431.568
Laplace	small / default / large	440.625 / 440.625 / 440.625

For this dataset and optimizer, both kernels converge to the same optimum across tested initial scales (small, default, large).

Effect of number of observations (L-BFGS-B, default init):

Kernel	frac	NLL	NLL/obs
SE	0.05	237.635	2.640
SE	0.10	431.568	2.398
SE	0.20	753.637	2.093
Laplace	0.05	256.989	2.855
Laplace	0.10	440.625	2.448
Laplace	0.20	747.771	2.077

As the training fraction increases, absolute NLL grows (more points) but NLL/observation decreases for both kernels, indicating better per-point fit.

Task 5 (implement posterior sampling and plotting functions). I implemented both required methods:

- `sample_from_posterior(trajectories=M)`: computes posterior mean/covariance from the trained GP and draws trajectories via Cholesky sampling.
- `plot_posterior()`: plots posterior trajectories, posterior mean, uncertainty band (± 2 std), and observations.

The posterior uses

$$\mu_{\star} = K_{\star x} K_{xx}^{-1} y, \quad \Sigma_{\star} = K_{\star\star} - K_{\star x} K_{xx}^{-1} K_{x\star}.$$

Task 6 (posterior plots and qualitative kernel differences). Figure 5 includes posterior samples, error bars (± 2 std), posterior mean, and observations for both kernels. Qualitatively, SE gives smoother posterior means; Laplace allows rougher/local changes and can track abrupt variations more readily.

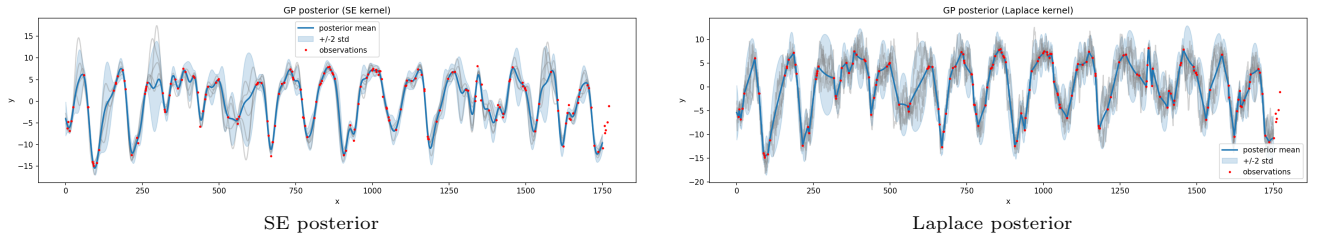


Figure 5: Part 2, Tasks 5–6: posterior predictions and uncertainty visualization for both kernels.

Task 7 (quantitative predictive assessment: NLL + MSE). With x the full centered series and $\hat{x}$ the posterior mean over all indices,

$$\text{MSE}(x, \hat{x}) = \frac{1}{N} \sum_{n=1}^N (x[n] - \hat{x}[n])^2.$$

I report NLL (from Task 4, 10% fit) and full-series MSE:

Kernel	Trained NLL (10% fit)	MSE on full series
Laplace	440.625	3.5493
SE	431.568	3.5916

In this run, SE is better by trained NLL while Laplace is better by full-series MSE, so evidence on observed points and reconstruction on the full signal do not necessarily rank kernels identically. As an additional uncertainty check, empirical coverage of the nominal 95% band ($\hat{x} \pm 2\sigma$) on the full series is 0.8839 for SE and 0.9656 for Laplace. This mismatch is expected because the metrics emphasize different aspects: NLL on the observed subset calibrated predictive distributions on sampled points, while full-series MSE rewards pointwise reconstruction over all indices (including unobserved ones during training). In this experiment, Laplace’s rougher prior/posterior family appears to fit local fluctuations better in MSE, while SE achieves slightly better likelihood on the observed subset. The coverage values are coherent with that interpretation: SE under-covers relative to nominal 95% (narrower uncertainty), whereas Laplace is closer to nominal calibration. For model selection, a robust conclusion should therefore consider all three diagnostics jointly (NLL, MSE, and coverage), rather than relying on a single scalar criterion.

Computational note: GP training scales as $O(N^3)$ due to Cholesky factorization of $K + \sigma_e^2 I$. The observed NLL/obs decrease with larger fractions is consistent with improved conditioning from more observations.

1

¹ Auditability/reproducibility: script-level seed is `np.random.seed(0)`; regenerate Part 2 outputs with `python3 solutions/part2/gaussian_process_submission.py`.

Part 3: Recovering EM from Variational Inference

Claim. Coordinate-ascent VI with variational family

$$q(z, \theta) = q_z(z) q_\theta(\theta), \quad q_\theta(\theta) = \delta(\theta - \theta_0),$$

recovers the standard EM algorithm.

Step 1: write ELBO exactly.

$$\mathcal{L}(q_z, \theta_0) = \mathbb{E}_{q_z(z)\delta(\theta-\theta_0)}[\log p(x, z, \theta) - \log q_z(z) - \log \delta(\theta - \theta_0)].$$

Integrating over θ by the delta mass gives

$$\mathcal{L}(q_z, \theta_0) = \int q_z(z) [\log p(x, z, \theta_0) - \log q_z(z)] dz + C_1,$$

where C_1 is constant w.r.t. q_z, θ_0 (it absorbs terms from $\log \delta$ and normalizing constants). Using $p(x, z, \theta_0) = p(x, z | \theta_0)p(\theta_0)$,

$$\mathcal{L}(q_z, \theta_0) = \underbrace{\int q_z(z) \log p(x, z | \theta_0) dz}_{\mathbb{E}_{q_z}[\log p(x, z | \theta_0)]} - \underbrace{\int q_z(z) \log q_z(z) dz}_{-H[q_z]} + C_2.$$

So, up to constants:

$$\mathcal{L}(q_z, \theta_0) = \mathbb{E}_{q_z}[\log p(x, z | \theta_0)] + H[q_z].$$

Step 2: optimize over q_z for fixed θ_0 (E-step). Solve

$$\max_{q_z} \mathbb{E}_{q_z}[\log p(x, z | \theta_0)] + H[q_z] \quad \text{s.t.} \quad \int q_z(z) dz = 1.$$

Lagrangian:

$$\mathcal{J}[q_z] = \int q_z(z) \log p(x, z | \theta_0) dz - \int q_z(z) \log q_z(z) dz + \lambda \left(\int q_z(z) dz - 1 \right).$$

Functional derivative:

$$\frac{\delta \mathcal{J}}{\delta q_z(z)} = \log p(x, z | \theta_0) - \log q_z(z) - 1 + \lambda = 0.$$

Hence

$$\begin{aligned} \log q_z(z) &= \log p(x, z | \theta_0) + (\lambda - 1), \\ q_z(z) &\propto p(x, z | \theta_0). \end{aligned}$$

Normalize:

$$q_z^*(z) = \frac{p(x, z | \theta_0)}{\int p(x, z' | \theta_0) dz'} = p(z | x, \theta_0).$$

This is exactly EM's E-step.

Step 3: optimize over θ_0 for fixed q_z (M-step). Terms depending on θ_0 :

$$\theta_0^* = \arg \max_{\theta_0} \mathbb{E}_{q_z}[\log p(x, z | \theta_0)].$$

If q_z is set to $p(z | x, \theta^{\text{old}})$, then

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}}), \quad Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{p(z|x, \theta^{\text{old}})}[\log p(x, z | \theta)],$$

which is the standard EM M-step. Therefore VI with a point-mass variational factor over θ is EM.

Part 4: α -Divergence Limits to KL Directions

Given

$$D_\alpha(p\|q) = \frac{4}{1-\alpha^2} \left(1 - \int p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}} dx \right), \quad \alpha \neq \pm 1.$$

Define

$$I(\alpha) = \int p(x)^{\frac{1-\alpha}{2}} q(x)^{\frac{1+\alpha}{2}} dx, \quad D_\alpha = \frac{4(1-I(\alpha))}{1-\alpha^2}.$$

Assume $p, q > 0$ on common support and integrability needed for first-order expansions.

A. Limit $\alpha \rightarrow 1$: obtain $\text{KL}(q\|p)$. Set $\alpha = 1 - \varepsilon$, $\varepsilon \downarrow 0$. Then

$$\frac{1-\alpha}{2} = \frac{\varepsilon}{2}, \quad \frac{1+\alpha}{2} = 1 - \frac{\varepsilon}{2}.$$

Hence

$$I(\alpha) = \int p^{\varepsilon/2} q^{1-\varepsilon/2} dx.$$

Use $a^\eta = \exp(\eta \log a) = 1 + \eta \log a + O(\eta^2)$:

$$p^{\varepsilon/2} = 1 + \frac{\varepsilon}{2} \log p + O(\varepsilon^2),$$

$$q^{1-\varepsilon/2} = q \exp\left(-\frac{\varepsilon}{2} \log q\right) = q \left(1 - \frac{\varepsilon}{2} \log q + O(\varepsilon^2)\right).$$

Multiply:

$$p^{\varepsilon/2} q^{1-\varepsilon/2} = q \left[1 + \frac{\varepsilon}{2} (\log p - \log q) + O(\varepsilon^2)\right].$$

Integrate:

$$I(\alpha) = 1 + \frac{\varepsilon}{2} \int q (\log p - \log q) dx + O(\varepsilon^2),$$

$$1 - I(\alpha) = \frac{\varepsilon}{2} \int q (\log q - \log p) dx + O(\varepsilon^2) = \frac{\varepsilon}{2} \text{KL}(q\|p) + O(\varepsilon^2).$$

Also

$$1 - \alpha^2 = 1 - (1 - \varepsilon)^2 = 2\varepsilon - \varepsilon^2 = 2\varepsilon + O(\varepsilon^2).$$

Therefore

$$D_\alpha = \frac{4 \left[\frac{\varepsilon}{2} \text{KL}(q\|p) + O(\varepsilon^2) \right]}{2\varepsilon + O(\varepsilon^2)} \xrightarrow{\varepsilon \rightarrow 0} \text{KL}(q\|p).$$

B. Limit $\alpha \rightarrow -1$: obtain $\text{KL}(p\|q)$. Set $\alpha = -1 + \varepsilon$, $\varepsilon \downarrow 0$. Then

$$\frac{1-\alpha}{2} = 1 - \frac{\varepsilon}{2}, \quad \frac{1+\alpha}{2} = \frac{\varepsilon}{2}.$$

So

$$I(\alpha) = \int p^{1-\varepsilon/2} q^{\varepsilon/2} dx.$$

Expand similarly:

$$p^{1-\varepsilon/2} = p \left(1 - \frac{\varepsilon}{2} \log p + O(\varepsilon^2)\right), \quad q^{\varepsilon/2} = 1 + \frac{\varepsilon}{2} \log q + O(\varepsilon^2).$$

Hence

$$p^{1-\varepsilon/2} q^{\varepsilon/2} = p \left[1 + \frac{\varepsilon}{2} (\log q - \log p) + O(\varepsilon^2)\right].$$

Integrating:

$$1 - I(\alpha) = \frac{\varepsilon}{2} \int p (\log p - \log q) dx + O(\varepsilon^2) = \frac{\varepsilon}{2} \text{KL}(p\|q) + O(\varepsilon^2).$$

Again $1 - \alpha^2 = 2\varepsilon + O(\varepsilon^2)$, so

$$D_\alpha \xrightarrow{\alpha \rightarrow -1} \text{KL}(p\|q).$$

Thus the two KL orientations are recovered as the endpoint limits of D_α .

AI acknowledgement. I used AI tools only to cross-check selected derivations and validate intermediate steps that I had already worked out independently. I also used them for routine code-level improvements (such as small refactoring and efficiency cleanups), while the modelling decisions, experiment design, result interpretation, and final write-up remained my own responsibility.